SWYNEX TECHNOLOGIES - CYBERSECURITY INTERNSHIP

TASK 3: WEB SECURITY REVIEW REPORT

Defensive Web Vulnerability Assessment, Risk Analysis & Remediation

Intern Name:	Ziyakausar Tamboli
Intern ID:	SWX-2026-002678
Program:	Cyber Security Internship (30 Days)
Target Application:	OWASP Juice Shop (v20.3.0)
Testing Environment:	Authorized Localhost Lab (http://localhost:3000)
Assessment Date:	November 2026

1. Executive Summary

This report details the defensive security review performed for Task 3 of the SWYNEX Cybersecurity Internship. Using the authorized local training environment (OWASP Juice Shop), five distinct security checks were conducted covering HTTP security headers, input sanitization (XSS), SQL injection authentication bypass, credential error handling, and endpoint access control.

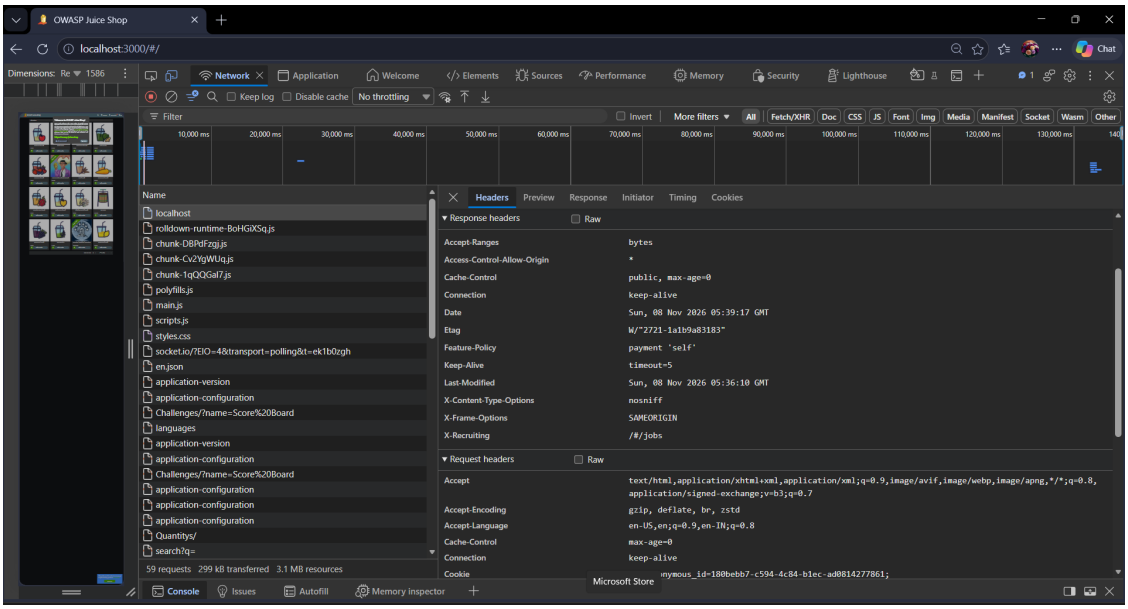
2. Security Tests & Findings Summary

Test	Test Name	Observed Result	Severity
Test 1	Security Misconfiguration - HTTP Headers	2 headers present; CSP & HSTS absent	**Medium**
Test 2	Cross-Site Scripting (XSS) Testing	Encoded in URL; no script execution in test	**Not Confirmed**
Test 3	SQL Injection - Authentication Bypass	Challenge solved; Admin bypassed via ' OR 1=1--	**High**
Test 4	Authentication Security Testing	Secure behavior; generic error on invalid login	**Low / Secure**
Test 5	Access Control Testing	Protected; unauthenticated whoami returns empty object	**Low / Secure**

3. Detailed Test Results & Evidence

Test 1: Security Misconfiguration – HTTP Security Headers Review

• Objective: To review HTTP response headers and identify security configuration observations.
• Observations: X-Content-Type-Options: nosniff and X-Frame-Options: SAMEORIGIN were present. CSP and HSTS were not visible in the response headers.
• Security Impact: Missing headers increase exposure to clickjacking or untrusted script execution.
• Remediation: Configure an appropriate Content-Security-Policy and enable HSTS on production HTTPS.
• Severity: Preliminary (Medium) | Evidence: 01_Security_Misconfiguration.png

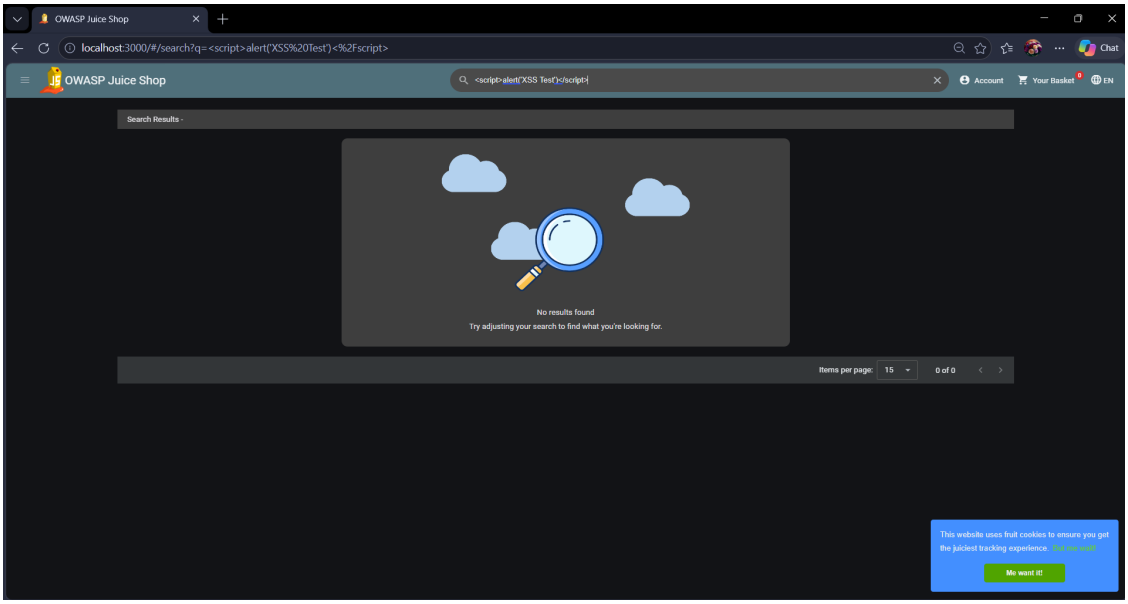


Evidence 1: HTTP Response Headers review in browser developer tools

---

Test 2: Cross-Site Scripting (XSS) Testing

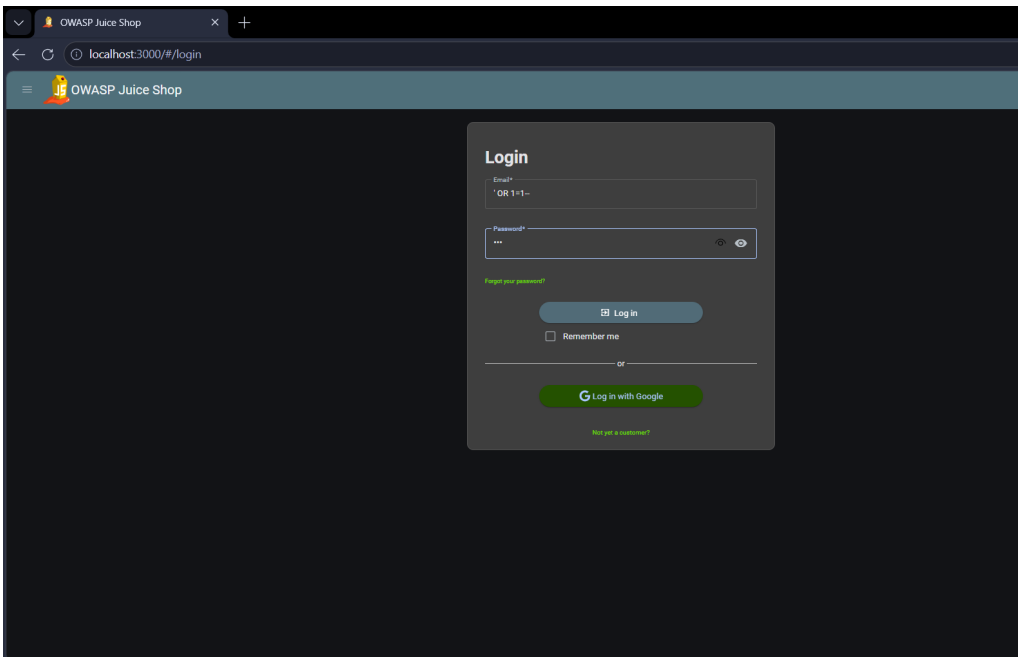
• Objective: To check whether the application safely handles user-supplied search input.
• Test Payload: <script>alert('XSS Test')</script>
• Observations: Displayed payload encoded in URL. Application displayed "No results found". No JavaScript alert execution visible in screenshot.
• Result & Severity: No XSS execution observed during this search test (Severity: Not confirmed).
• Remediation: Apply context-aware output encoding and validate/sanitize untrusted user inputs.
• Evidence: 02_XSS_Testing.png



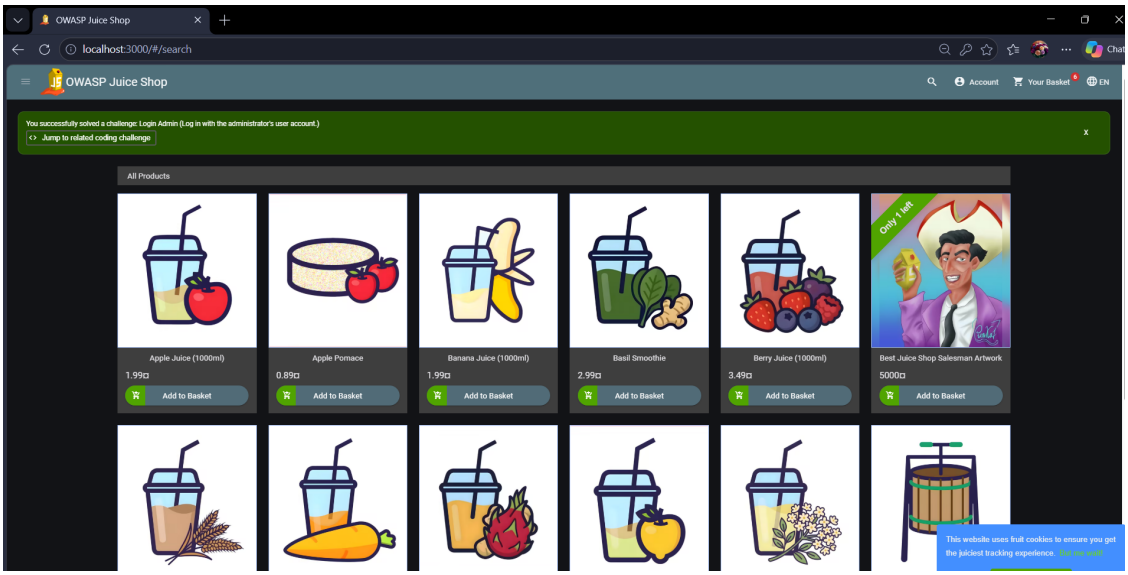
Evidence 2: Search input field testing with script payload

Test 3: SQL Injection – Authentication Bypass

• Objective: To evaluate whether unsafe SQL query handling allows unauthorized authentication.
• Testing Method: Injected test payload ' OR 1=1-- into the email login field.
• Observed Result: The application displayed the successful "Login Admin" challenge notification. Administrator authentication was successfully obtained.
• Security Impact: Total authentication bypass allowing full access to administrative features.
• Severity: High | Evidence: 03A_SQL_Injection_Payload.png, 03B_SQL_Injection_Success.png
• Remediation: Enforce parameterized SQL queries and prepared statements; eliminate raw concatenation.



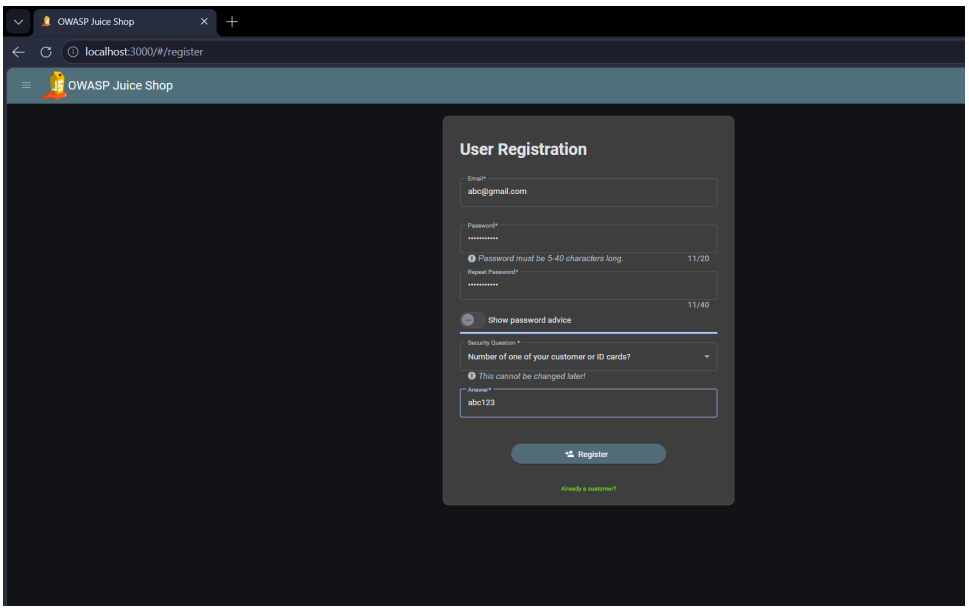
Evidence 3A: Submitting SQL injection payload into login form



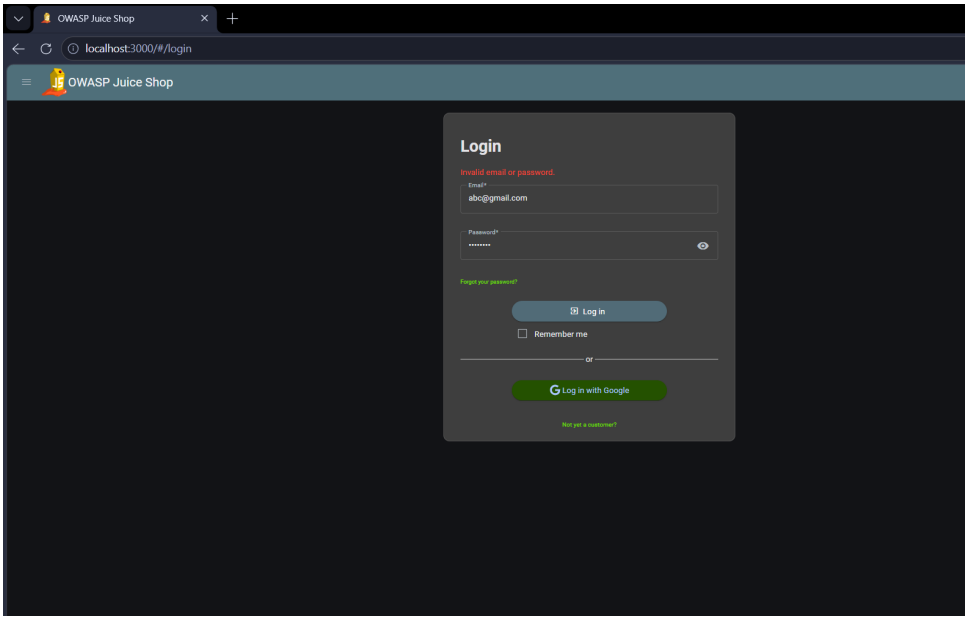
Evidence 3B: Successfully solved "Login Admin" challenge notification banner

Test 4: Authentication Security Testing

• Objective: To evaluate application handling of invalid login credentials.
• Testing Method: Registered test user (abc@gmail.com) and attempted login with incorrect password.
• Expected & Observed Result: Application properly rejected the attempt with generic "Invalid email or password" error, preventing user enumeration.
• Severity: Low / Informational (Secure Error Handling Verified).
• Remediation: Maintain consistent error messages, implement rate limiting / lockout mechanisms.
• Evidence: 04A_Authentication_Registration.png, 04B_Authentication_Invalid_Credentials.png



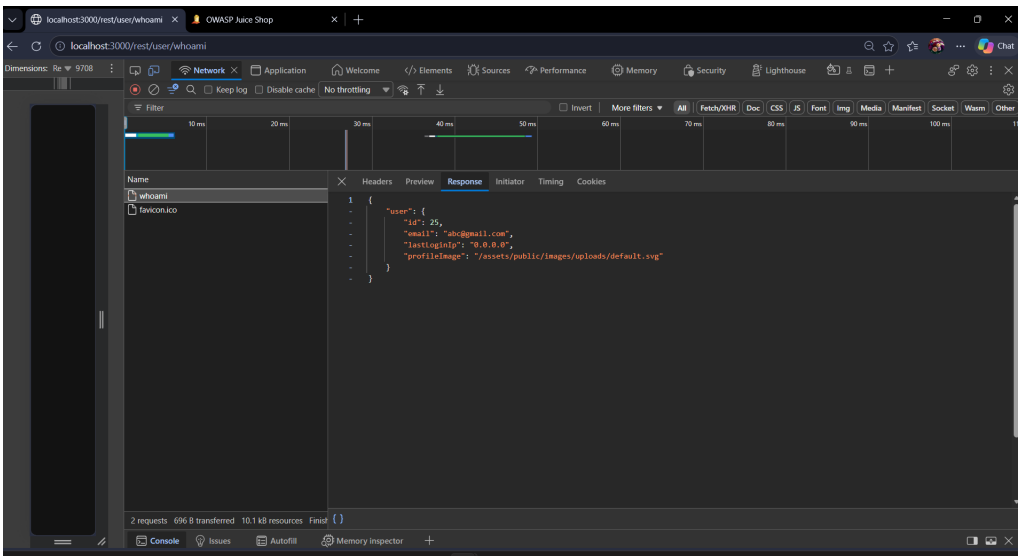
Evidence 4A: Baseline user account registration



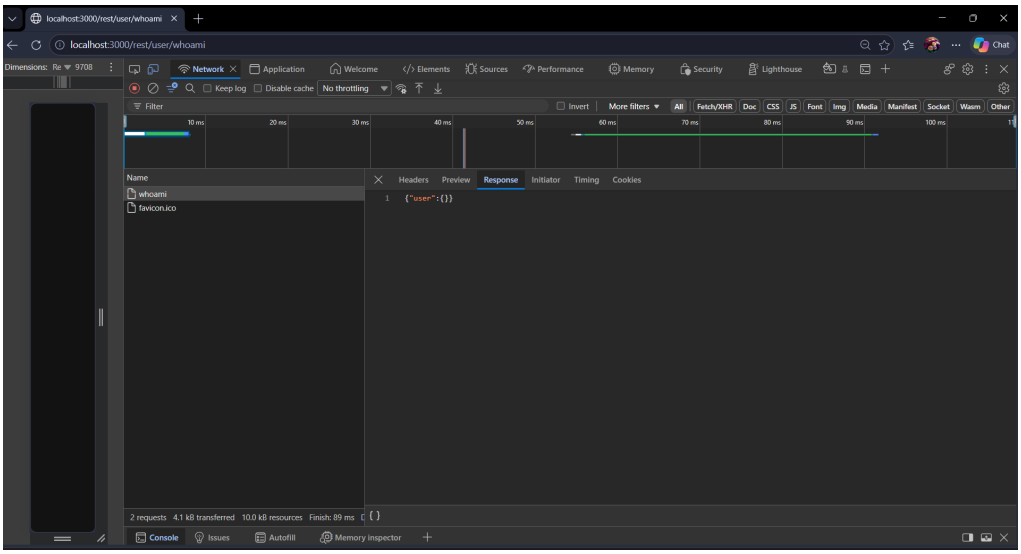
Evidence 4B: Rejection with generic error "Invalid email or password"

Test 5: Access Control Testing

• Objective: To verify whether unauthorized users can access protected account information.
• Testing Method: Accessed /rest/user/whoami when logged in vs logged out and compared responses.
• Observed Result: When authenticated, returns current user object (id, email). When logged out, returns empty object {"user":{}}, withholding sensitive user data.
• Severity: Low / Verified Secure.
• Remediation: Maintain server-side token verification and apply role-based access control (RBAC).
• Evidence: 05A_Access_Control_LoggedIn.png, 05B_Access_Control_LoggedOut.png



Evidence 5A: Authenticated response containing user identity details



Evidence 5B: Unauthenticated logged-out response withholding user data